

WinnerRadar — Master Technical Documentation & Onboarding Guide

Audience: A Senior Software Engineer joining the team today with zero prior context. **Goal:** After reading this, you can navigate, run, debug, and extend every part of the system. **Generated:** 27–29 June 2026 · Repository: `abdelbadiekertimi1212-max/spytool` · Branch: `main`

Table of Contents

- 1. Complete System Overview & Architecture
 - 2. Exhaustive Deconstruction of the Engines & Backend
 - 3. Frontend & UI/UX Mechanics
 - 4. Database, RLS & Security Posture
 - 5. The Bug & Bottleneck Audit (System Health)
 - 6. Autonomous Innovation & Free Open-Source Roadmap
- Appendix A — File Map
 - Appendix B — Environment Variables
 - Appendix C — Commands & Scripts

1. Complete System Overview & Architecture

1.1 What this SaaS is (business)

WinnerRadar is a market-intelligence ("spy") tool for the **Algerian Cash-on-Delivery (COD) e-commerce market**. Its core promise: surface **"Winning Products"** — items selling fast *and* backed by active paid advertising — **before they saturate**, so dropshippers/merchants can source and launch them early.

A secondary product is a **B2B Lead-Generation CRM**: every discovered store is also a sales lead (for freelancers/agencies selling media-buying & fulfillment services), enriched with an AI `lead_score` and one-click AI cold-outreach.

1.2 The core thesis (the single most important idea)

Confirmed Winner = (Inventory Depletion Velocity) × (Active Meta Ad Verification).

Neither signal alone is trusted. Fast stock drops without ads could be a manual inventory edit; ads without sell-through is just spend. The whole system is built to compute the **intersection** of these two axes, with strict anti-false-positive guards.

1.3 Tech stack (every layer)

Layer	Technology	Notes
Framework	Next.js 14.2.35 (App Router) + React 18 + TypeScript	Server Components by default
Styling/UI	Tailwind CSS 3.4 , shadcn/ui (Radix primitives), Framer Motion 12	Dark theme default ("cinema" aesthetic)
i18n	next-intl v4	ar (RTL, default), fr, en
DB / Auth	Supabase (Postgres + Auth + RLS) via <code>@supabase/ssr 0.12</code>	Project ref <code>ywfainbmubmuoswbmoh</code>
Inventory scraping	Crawlee 3.17 (<code>CheerioCrawler</code>) + native <code>fetch</code>	Shopify/YouCan/Storeino
Ads + Discovery scraping	Playwright + <code>playwright-extra</code> + <code>puppeteer-extra-plugin-stealth</code>	Public Meta Ad Library (no official API token)
AI	Groq SDK → <code>llama-3.3-70b-versatile</code>	Niche classification + outreach copy
Payments	Chargily Pay V2 (REST, DZD / Edahabia & CIB)	<code>/api/checkout</code> + webhook
Email	Resend	Cold-outreach send
Rate limiting	Upstash Ratelimit + Redis , with in-memory fallback	<code>/api</code> protection
Charts	Recharts 3	Analytics dashboard
Script runner	tsx	Standalone engine scripts
Scheduling	GitHub Actions cron (every 6h)	<code>.github/workflows/engine.yml</code>
PDF/docs	marked + Playwright (this document)	<code>scripts/gen-docs-pdf.mjs</code>

1.4 Two runtimes, one repository

The codebase is split into two cooperating runtimes:

1. **The Next.js web app** (`app/` , `components/` , `lib/` minus `lib/engine`) — what users see; runs on Vercel/Node; talks to Supabase with the **anon key** under RLS.
2. **The Engine** (`lib/engine/` , `scripts/`) — headless data pipeline; runs on **GitHub Actions** via `tsx` ; talks to Supabase with the **service-role key** (bypasses RLS). Loads env via `lib/engine/load-env.ts` .

They share one thing: the **typed Supabase schema** (`types/supabase.ts`) and the database itself.

1.5 Request & data lifecycles (traced end-to-end)

A) Authentication / session

1. User submits the login form (`components/auth/login-form.tsx` , client) → `supabase.auth.signInWithPassword()` (browser client, `lib/supabase/client.ts`).
2. Supabase sets the session cookie. On every subsequent request, `middleware.ts` runs `updateSession()` (`lib/supabase/middleware.ts`) which refreshes the cookie **and** runs the next-intl locale middleware in the same pass.
3. `app/[locale]/dashboard/layout.tsx` (server) calls `supabase.auth.getUser()` ; if no user → `redirect('/{locale}/login')` . This is the **client-facing auth gate**.

B) Viewing the Winner feed

1. `app/[locale]/dashboard/page.tsx` (server, `force-dynamic`) runs **two queries in parallel**: `products` joined with `store(*, ads(*))`, and `getDashboardAnalytics()` (OSINT aggregates).
2. Data is passed as props to client components: `<AnalyticsOverview>` (Recharts) and `<WinnerFeed>`.
3. `<WinnerFeed>` does **100% client-side filtering/sorting** with `useMemo` (`search`, `niche`, `platform`, `velocity`, `ads`, `date`, `sort`). No further network calls while filtering.

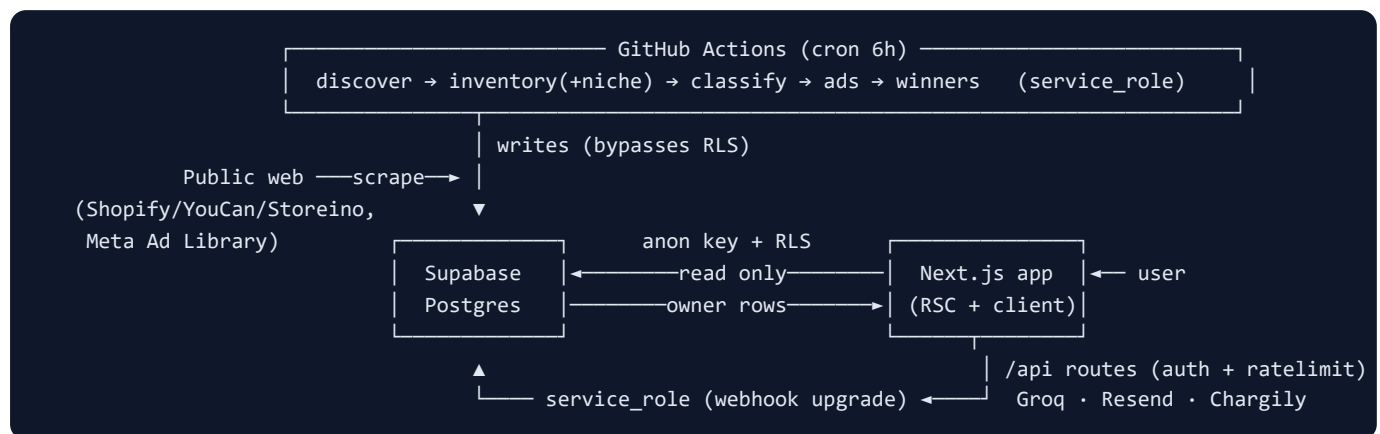
C) AI Cold Outreach (CRM)

1. `app/[locale]/dashboard/leads/page.tsx` (server) loads stores + aggregates `winner_count/max_velocity/active_ads` per store → `<LeadsTable>`.
2. User clicks **Outreach** → `<OutreachModal>` → `POST /api/outreach {storeId, locale}`.
3. Route (`app/api/outreach/route.ts`): `getUser()` → `rate-limit` → load store + its winners → `generateOutreach()` (`lib/groq.ts`, Groq JSON mode) → returns `{subject, body, callHook}`.
4. User edits, clicks **Send** → `POST /api/outreach/send` → `rate-limit` → `sendOutreachEmail()` (`lib/resend.ts`). Also offers `tel:` and `mailto:` fallbacks.

D) Payment / tier upgrade

1. `<Pricing>` → `POST /api/checkout {tier}` → `auth + rate-limit` → `createCheckout()` (`lib/chargily.ts`) with `metadata:{user_id,tier}` → returns `hosted_checkout_url` → client redirects.
2. After payment, Chargily calls `POST /api/webhooks/chargily` → `verifyWebhookSignature()` (HMAC-SHA256) → on `checkout.paid`, the **service-role admin client** updates subscriptions (`status active`, `tier`, `+30 days`).

E) The Engine pipeline (cron, every 6h) `discover` → `inventory` → `classify` → `ads` → `winners`. Each is a `tsx` script using the **service-role** client. Detail in §2.



2. Exhaustive Deconstruction of the Engines & Backend

All engine code lives in `lib/engine/`. Shared infra:

- **config.ts** — every tunable, env-overridable via `num(NAME, fallback)`. Key values: `maxProductsPerStore=60`, `minDelayMs=800`, `maxDelayMs=2600`, `maxConcurrency=3`, `requestTimeoutMs=25000`, `shopifyProbeQuantity=99999`. Winner: `windowDays=14`, `minDailyVelocity=3`, `minActiveAds=1`, `maxSaleDropPerWindow=100`, `minAdAgeDays=3`, `minDistinctCreatives=2`, `consensusBoost=25`. Meta: `searchCountry=DZ`, `maxAdsPerStore=30`, `maxScrolls=6`, `navTimeoutMs=45000`, `headless=true`. Discover: default keywords (الدفع عند الاستلام, التوصيل مجاني, 58 ولاية, الدفع عند الاستلام), `maxCandidatesPerKeyword=40`.
- **http.ts** — `jitter()` (randomized delay), `randomUserAgent()` (rotates a 4-UA pool), `stealthHeaders()`, `fetchWithTimeout()` (AbortController + allowed-status list), `sanitizeText()` (strips control chars / lone UTF-16 surrogates so PostgREST JSON insert never breaks), `originOf()`, `toNumber()`.
- **supabase.ts** — `createEngineClient()` = service-role client (`persistSession:false`).

- `logger.ts` — `logEngine(client, level, scope, message, context)` writes to `engine_logs` **and** console; never throws.
- `text.ts` — `normalizeTitle()` (lowercase, strip ASCII+Arabic punctuation, collapse whitespace; keeps Arabic letters — avoids `\w`). Shared by `winner` & `osint` to avoid a circular import.

2.1 The Discovery Engine — `discover.ts` (`engine:discover`)

Self-feeding store finder. Steps:

1. Load existing store hostnames (dedup set).
2. For each broad keyword, `MetaAdLibraryScraper.searchAdLinks(kw)` (Playwright) opens the Ad Library, scrolls, and harvests candidate URLs from each ad card via in-page `extractAdLinks()` : decoded `1.facebook.com/1.php?u=` outbound links + visible caption domains (regex over `innerText` for `domain.(com|store|shop|dz|...)`).
3. Normalize to hostnames; drop a **denylist** (facebook, instagram, tiktok, youtube, google, link shorteners, etc.) and anything already tracked. Cap `maxCandidatesPerKeyword` per keyword.
4. For each new candidate, `detectPlatform(url)` :
 - **Fast path:** host suffix (`*.myshopify.com` , `*.youcan.store/.shop` , `*.storeino.store/.com`).
 - **Deep:** fetch the homepage, inspect final URL + HTML + headers for signatures (`cdn.shopify.com` , `/cdn/shop/` , `Shopify.theme` , `x-shopify-stage` ; `cdn.youcan.shop` , `youcanjs` ; `storeino`).
 - **Shopify fallback:** probe `/products.json?limit=1` — valid `{products:[]}` JSON $\Rightarrow$ Shopify.
5. Insert new valid stores (`is_active=true`). Result counts logged; errors go to `engine_logs` via `logEngine` .

Live proof: keyword 121 $\rightarrow$ الدفع عند الاستلام links $\rightarrow$ 35 candidates $\rightarrow$ 9 new stores inserted (6 YouCan, 3 Shopify).

2.2 The Inventory Engine — `scrape-inventory.ts` + `shopify.ts` / `youcan.ts` / `storeino.ts`

(`engine:inventory`)

For each active store, `scrapeStore(target)` (`index.ts`) routes by platform:

Shopify (`shopify.ts`)

1. Page through public `GET /products.json?limit=250&page=N` (cap 10 pages). Extract id/title/handle/price/compare-at/image/url from each product's first variant.
2. **Stock via the "threshold hack"** (`probeShopifyStock`): `POST /cart/add.js` with `quantity: 99999` . Shopify replies **HTTP 422** with "You can only add N ... to the cart." $\rightarrow$ regex the integer N = real available stock. HTTP 200 $\Rightarrow$ untracked/unlimited $\Rightarrow$ `stock=null` . Probes are capped (`maxProductsPerStore`) with `jitter()` between calls.

YouCan / Storeino (`crawl.ts` + `extract.ts`)

1. `crawlStorefront()` uses `Crawlee CheerioCrawler` (in-memory storage, `maxConcurrency=3` , stealth headers via `preNavigationHooks`), starting at the store URL and enqueueing product-looking links (`/products/**` , `/product/**` , `/p/**` , `/item/**`).
2. On each product page, `extractProduct()` and `extractStock()` apply **platform default-theme selectors** first (e.g. YouCan `h1.product-title` , `input[name='quantity'][max]`), then generic fallbacks: inline JSON keys (`available_quantity` , `inventory_quantity` , `quantity` , `stock` , `qty`), quantity-input `max` , data-attributes, then JSON-LD `offers.availability` (boolean only).

Persistence (`persistence.ts` > `persistScrape`)

1. Upsert products on conflict (`store_id` , `external_id`) ; sanitize text.
2. Seed `initial_stock` the first time a known stock is seen.
3. Append a **product_snapshots** row per product (the time-series that powers velocity).
4. **AI niche tagging** of newly inserted products (those with `niche IS NULL`) via `tagProductsByNiche` (non-fatal).
5. After the full run, `purgePlaceholders()` deletes junk (titles `test / test product /Arabic equivalents`, or `price = 0`).

2.3 The Ad Verification Engine — meta-ads.ts + fetch-ads.ts (engine:ads)

MetaAdLibraryScraper (Playwright + stealth, one shared browser):

1. search(term) opens facebook.com/ads/library/?active_status=active&country=DZ&q=<term>&search_type=keyword_unordered, dismisses cookie consent, waits for Library ID, scrolls maxScrolls times.
2. In-page extractAdCards() anchors on the stable "Library ID: <digits>" label, climbs to a card ancestor (stopping before sibling cards merge), and extracts: advertiser name, ad copy (prefers Arabic leaf text, longest non-noise block), creative media (<video src|poster> or large scontent/fbcdn), and run date (parseStartDate).
3. Each store's best search term = fb_page_name or a name derived from its domain. Results are persisted by persistAds, which **flips all of a store's ads to is_active=false first**, then upserts the freshly-found active ads (so is_active stays truthful).

Key constraint (documented intentionally): the **official** Ad Library API returns only an ad_snapshot_url for commercial ads, not media. That's why we **scrape the public site with Playwright** instead — it yields the real creative mediaUrl. The dashboard renders that as a static thumbnail with a fallback poster (fbcdn is hotlink-protected; see §3.4).

2.4 The Classifier Engine — classifier.ts + classify-products.ts (engine:classify)

- Fixed 16-niche taxonomy (Automotive Accessories, Women's/Men's Fashion, Kitchen Gadgets, Beauty, ..., Uncategorized).
- classifyBatch(items) : **one** Groq request per ≤25 products, temperature:0, JSON mode, returns {results: [{i,niche}]} . Output is coerced against the taxonomy (anything off-list → Uncategorized).
- classifyUntagged(client, max) : selects niche IS NULL and tags them.
- classifyAll(client, {chunkSize,delayMs}) : **loops in chunks with a delay** until the queue is empty (rate-limit friendly) — this is what engine:classify calls. Degrades to a no-op if GROQ_API_KEY is unset.

2.5 The Winner Algorithm — winner.ts (engine:winners)

(a) computeVelocity(snapshots, maxSaleDrop=100) — **Smart Velocity** Sorts snapshots ascending; for each consecutive pair computes drop = prev.stock - curr.stock.

- A drop counts as **real sales only if** $0 < \text{drop} < \text{maxSaleDrop}$. Drops ≤ 0 (restock) or ≥ 100 (manual inventory reset) are **excluded from both** the sold-units sum and the elapsed-time denominator.
- soldUnits = Σ counted drops ; countedDays = Σ counted intervals / 86,400,000 ms.
- velocity = soldUnits / countedDays (0 if < 2 snapshots or countedDays ≤ 0).

(b) computeWinners(client) — **The 3D Verification**

1. Build per-store ad strength from active ads: distinctCreatives (unique ad_creative_url / meta_ad_id) and earliestStart (oldest active-ad start date).
 - adCommitmentOK(store) = earliestStart older than minAdAgeDays(3) **AND** distinctCreatives $\geq$ minDistinctCreatives(2).
2. Build **market consensus**: normalizeTitle(title) → Set<store_id> across all products (paged 1000).
3. For each product (paged 500), **bulk-fetch** its window snapshots (one .in('product_id', batchIds) query per batch — the N+1 fix), then:
 - velocityOK = velocity $\geq$ minDailyVelocity(3)
 - is_winner = velocityOK **AND** adCommitmentOK(store)
 - consensus = (#stores selling this normalized title) > 1
4. Update each product's daily_velocity, total_sold, is_winner, winner_since.
5. **Axis 3 (lead-score boost):** for stores scaling consensus winners, set lead_score = clamp(0..100, consensusWinners×25 + min(distinctCreatives×5, 20)).

2.6 The OSINT Engine — `osint.ts`

Powers the analytics dashboard + competitor intelligence (all server-side, take a Supabase client):

- `marketShareByPlatform()` — active store count per platform (pie chart).
- `priceDistribution()` — DZD histogram buckets (<1k ... 8k+) + average price.
- `computeMarketSaturation()` — group products by `normalizeTitle`, count **distinct stores per product**; returns those sold by ≥ 2 stores (contested/saturated).
- `computeAdStrength()` — **score = daysActive × activeAdCount** per store (a free proxy for confirmed ad spend, since real budgets are private).
- `computeInventoryDeltaForProduct()` — units sold/day for one product from its snapshot history (reuses Smart Velocity).
- `getDashboardAnalytics()` — runs the totals + all of the above in parallel for the dashboard.

2.7 Scheduling & background work

`.github/workflows/engine.yml`: cron `0 */6 * * *` (every 6h) + manual `workflow_dispatch`; concurrency: `winner-engine` with `cancel-in-progress:false` (no overlap → no IP bans); 30-min timeout; **Node 24** (npm 11, to match the lockfile generator); steps: checkout → setup-node → `npm ci` → install Playwright Chromium → discover → inventory → classify → ads (`continue-on-error`) → winners. Secrets: `NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY` (+ optional `GROQ_API_KEY`).

3. Frontend & UI/UX Mechanics

3.1 Routing & i18n

- App Router with a single dynamic segment `app/[locale]/...`. `i18n/routing.ts` defines locales `['ar', 'fr', 'en']`, default `ar`, `localePrefix:'always'`. `getDirection()` returns `rtl` for `ar`.
- `app/[locale]/layout.tsx` sets `<html lang dir>` + `dark` class + fonts, wraps children in `NextIntlClientProvider`, and calls `setRequestLocale()` for static rendering.
- `middleware.ts` composes the next-intl middleware with Supabase session refresh. Matcher excludes `api`, `_next`, static files.
- Navigation uses locale-aware `Link` / `useRouter` from `i18n/navigation.ts`.

3.2 Server↔client component split

- **Server Components** (pages/layouts) do all Supabase reads via `lib/supabase/server.ts` (cookie-bound, RLS-scoped) and pass plain props down.
- **Client Components** (`"use client"`) handle interactivity: feed filtering, charts, modals, the locale switcher, auth form, pricing. They talk to the backend only through `/api routes` (outreach, checkout) using `fetch`.

3.3 State management

- No global store (no Redux/Zustand). State is **local** `useState` + `useMemo` per component. The feed's filter/sort is a pure `useMemo` over the props array — instant, zero network.
- Server state is fetched per-request in Server Components (`force-dynamic` on authed pages).

3.4 Notable UI mechanics

- **Analytics** (`analytics-overview.tsx`): Recharts `PieChart` / `BarChart` in `ResponsiveContainer`; fills use CSS vars (`hsl(var(--winner))`) so they theme correctly; KPI cards + OSINT leaderboards.

- ### 3.5 Auth gate & "paywall" on the client

- ## 4. Database, RLS & Security Posture

1. `20260626000001_init_schema.sql` — extensions, enums, tables, triggers, indexes, original RLS (subscription-gated catalog).
2. `20260627010000_add_product_niche.sql` — `products.niche` + index.

3. `20260627020000_hardening_rls_indexes.sql` — high-volume indexes, `bookmarks`, `engine_logs`, and the **current** RLS policy set.

4.3 RLS policies in plain English (current, post-hardening)

- **profiles** — a user can **read and update only their own** row (`auth.uid() = id`). No inserts (the signup trigger handles that). No deletes.
- **subscriptions** — a user can **read only their own** subscription. All writes happen via the service-role Chargily webhook.
- **stores / products / product_snapshots / ads** — **any authenticated user can READ everything** (`using (true)`). There are **no insert/update/delete policies**, so clients cannot write; only the **service-role** engine (which bypasses RLS) writes.
- **bookmarks** — a user can **read/insert/delete only their own** bookmarks.
- **engine_logs** — **no client policies at all** → only the service-role can read/write it.

How the paywall is supposed to work vs reality: the original design gated catalog reads on `private.has_active_subscription()`. The Production-Hardening audit (per the spec "authenticated users can READ global data") replaced that with `using (true)`. **Net effect: the paywall is currently disabled at the DB level.** To re-enable, change the four catalog `select` policies from `using (true)` → `using (private.has_active_subscription())`.

4.4 Security posture summary

- ☒ Service-role key only in engine/webhook (server). `lib/supabase/admin.ts` is guarded with `import "server-only"`.
- ☒ `.env.local` is gitignored & untracked; repo scan finds no hardcoded secrets.
- ☒ Rotated User-Agents on all scrapers.
- ☒ HMAC-SHA256 verification on the Chargily webhook (timing-safe compare).
- ☒ Rate limiting on `/api/outreach`, `/api/outreach/send`, `/api/checkout` (Upstash, with in-memory fallback).
- ⚠️ See §5 for the open items (paywall, in-memory limiter on serverless, image host allowlist, secret rotation).

5. The Bug & Bottleneck Audit (System Health)

Ordered by severity. Each item is something the new engineer can pick up immediately.

Critical / business-impacting

1. **The subscription paywall is not enforced anywhere.** RLS catalog policies are `using (true)` and `dashboard/layout.tsx` checks only `getUser()` (not tier). Any authenticated free/expired user sees all data. **Fix:** re-add `private.has_active_subscription()` to catalog policies **and/or** gate the dashboard layout/feed on `subscriptions.status`.
2. **Rate limiting is weak on serverless.** The in-memory fallback (`lib/ratelimit.ts`) is per-process; on Vercel serverless each invocation can be a cold process, so the fallback barely limits. **Fix:** configure real `UPSTASH_REDIS_REST_URL/TOKEN` (free tier) so the sliding window is shared.

Performance / scaling

3. **computeWinners write amplification.** Snapshots are now bulk-fetched (good), but the function still issues **one UPDATE per product** plus a full title scan each run — $O(N)$ round-trips (~2,559 products → thousands of writes; the live run timed out at 9 min). **Fix:** batch updates (e.g. `upsert` of computed columns, or a single SQL `update ... from (values ...)`), or move the whole computation into a Postgres function / materialized view.
4. **Niche backfill latency.** `engine:classify` is sequential Groq calls; fine for cron but slow for large backlogs. Consider parallelism with a concurrency cap.

5. **Next/Image is effectively disabled.** `next.config.mjs` sets `remotePatterns: hostname: '*'` + components use `unoptimized /raw <img>`. No optimization, no domain allowlist (minor SSRF/abuse surface, large payloads). **Fix:** re-host scraped images to Supabase Storage via `sharp` and serve optimized.

Correctness / data quality

6. **Real winners need multiple cycles.** Velocity requires ≥ 2 snapshots over time; a freshly scraped product has one snapshot $\rightarrow$ velocity 0 $\rightarrow$ never a winner until the cron runs across days. *By design*, but a new engineer will be confused that "winners=5" are only the seeded demo. Document/seed expectations.
7. **Ad $\leftrightarrow$ product mapping is store-level.** `ads.product_id` is usually `NULL`; "this product has active ads" is inferred from the **store** having ads. This can over-credit ad backing to non-advertised products in the same store.
8. **YouCan/Storeino extraction is theme-dependent.** Selectors target default themes; custom-themed `.dz` stores will yield many `null` titles/prices/stock. Needs per-store tuning or a headless-render fallback.
9. **Stat niche breakdown undercounts** (`scripts/stats.ts`) due to the PostgREST 1000-row default cap — cosmetic, but misleading.
10. **Webhook replay/idempotency.** `checkout.paid` extends the period each time it's received; a duplicate/replayed (validly-signed) event would re-extend. Add an idempotency guard (store processed event ids).

Resilience / scraping

11. **Meta Ad Library brittleness & ban risk.** DOM heuristics break when Facebook obfuscates; a single browser with no proxy rotation risks IP bans at scale. **Mitigation:** proxy rotation + session pool + backoff (see §6).
12. **Shopify probe assumptions.** `cart/add.js` thresholding only reflects stock when inventory is *tracked*; many stores show `null`. Acceptable, but velocity then relies on stores that track inventory.

Engineering hygiene

13. **No automated tests and no PR CI** (only the engine cron). A typecheck/build/test gate would prevent regressions like the recent `npm ci` lockfile break.
14. **Secrets were shared in chat during setup** $\rightarrow$ rotate the GitHub PAT, Supabase DB password, and service-role/key set.
15. `/api/remix-ad` is a `501` scaffold (intentional placeholder for the video phase).

6. Autonomous Innovation & Free Open-Source Roadmap

Constraint honored: every suggestion below uses **100% free, open-source, self-hostable** tools. No paid APIs.

6.1 Smarter, ban-resistant scraping

- `pg-boss` (OSS, runs on the Supabase Postgres you already have) $\rightarrow$ replace ad-hoc script ordering with a real **job queue** (retries, scheduling, dead-letter). Zero new infra.
- **Crawlee** `ProxyConfiguration` + `SessionPool` (already a dep) + free rotating IPs (e.g. self-hosted via `proxy-chain`) $\rightarrow$ rotate identities, auto-retire blocked sessions.
- `robots-parser` (OSS) $\rightarrow$ respect robots.txt per store (compliance + politeness).
- **got-scraping** browser-fingerprint headers (bundled with Crawlee) for the non-Playwright fetches.
- Rotate Playwright **browser engines** (chromium/firefox/webkit) and add human-like mouse/scroll jitter to further reduce Ad Library detection.

6.2 Local / free AI (drop the Groq dependency entirely if desired)

- **Ollama** (OSS, self-host `llama3 / qwen2`) $\rightarrow$ run niche classification & outreach generation locally, free, no rate limits.
- `@xenova/transformers` (**Transformers.js**) (OSS) $\rightarrow$ run sentence-embedding models in Node/edge with no API.

- `pgvector` (Supabase-native extension) + those embeddings → **semantic product de-duplication**: cluster "same product across stores" by meaning, not just normalized title. This dramatically improves Market Saturation and consensus accuracy.
- **Perceptual image hashing** (`sharp` + `blockhash-core` , OSS) → detect the *same product image* reused across stores even when titles differ — a powerful "who's copying whom" signal.

6.3 Richer data & visualization

- `@tanstack/react-table` (OSS) → sortable/filterable/paginated CRM table (replaces the static table).
- `@tanstack/react-query` (OSS) → client caching, background refetch, and **optimistic UI** for bookmarks/outreach.
- **Apache ECharts** or `@nivo` (both OSS) → heatmaps, treemaps (niche share), and velocity trend lines beyond Recharts.
- **Full-text search**: Postgres `tsvector` + the existing `pg_trgm` → instant server-side product search instead of client-side filtering as the dataset grows.

6.4 User retention & growth (all free/OSS)

- **Supabase Realtime** (free) + `bookmarks` (table already exists) → **"Winner Alerts"**: notify a user the moment a product in their saved niche becomes a confirmed winner.
- `react-email` (OSS) → beautiful templated **weekly winner digests** (sent through the existing Resend integration, or self-hosted SMTP).
- `next-pwa` + `web-push` (OSS, VAPID is free) → installable mobile PWA with push notifications.
- `date-fns` (OSS) → trend windows ("new winners this week", 7/30-day deltas).

6.5 Reliability & ops

- `vitest` + `@playwright/test` (OSS) → unit tests for the winner/velocity math and e2e for auth/feed; add a **GitHub Actions PR workflow** running `typecheck` → `lint` → `build` → `test` .
- `pino` (OSS) structured logging in the app/API layers, complementing the `engine_logs` table; build a `/dashboard/health` page that reads `engine_logs` so failures are visible without the terminal.
- `fluent-ffmpeg` + `ffmpeg-static` (OSS) → implement the scaffolded `/api/remix-ad` (download competitor ad video → trim/re-crop/watermark → upload to Supabase Storage).

6.6 Suggested first sprint for the new engineer

1. Re-enable the paywall (RLS + layout tier check) — *business critical*.
 2. Configure Upstash (free) so rate limiting is real on serverless.
 3. Batch `computeWinners` updates (single SQL) — kills the biggest bottleneck.
 4. Add a PR CI workflow (typecheck/build) to prevent lockfile/regression breaks.
 5. Prototype `pgvector` semantic dedup to upgrade Market Saturation.
-

Appendix A — File Map

```
app/[locale]/          layout (html/dir/i18n), landing page, login, legal/{privacy,terms}
app/[locale]/dashboard/ layout (auth gate), page (feed+analytics), leads, billing
app/api/               checkout, outreach, outreach/send, webhooks/chargily, remix-ad
components/ui/         shadcn primitives (button, card, dialog, table, ...)
components/dashboard/  winner-feed, winner-card, ad-creative, analytics-overview,
                        margin-calculator, leads-table, outreach-modal, pricing
components/auth/       login-form, sign-out-button
components/landing/    hero
lib/supabase/          client, server, admin (server-only), middleware
lib/engine/            config, http, supabase, logger, text, types, index,
                        discover, shopify, youcan, storeino, crawl, extract,
                        meta-ads, classifier, winner, osint, persistence, load-env
lib/                   groq, resend, chargily, billing, ratelimit, format, utils
lib/dashboard/         types (WinnerProduct, LeadRow)
scripts/               discover-stores, scrape-inventory, fetch-ads, classify-products,
                        compute-winners, stats, seed-*, wipe-demo
supabase/migrations/  3 migrations (init, niche, hardening)
i18n/ · messages/      routing/request/navigation · ar.json/fr.json/en.json
.github/workflows/     engine.yml (cron pipeline)
types/supabase.ts      hand-authored typed Database
```

Appendix B — Environment Variables

`NEXT_PUBLIC_SUPABASE_URL` , `NEXT_PUBLIC_SUPABASE_ANON_KEY` , `SUPABASE_SERVICE_ROLE_KEY` , `SUPABASE_PROJECT_REF` ,
`SUPABASE_DB_PASSWORD` , `META_SEARCH_COUNTRY` , `GROQ_API_KEY` , `GROQ_MODEL` , `RESEND_API_KEY` , `RESEND_FROM_EMAIL` ,
`CHARGILY_SECRET_KEY` , `CHARGILY_WEBHOOK_SECRET` , `CHARGILY_MODE` , `UPSTASH_REDIS_REST_URL` , `UPSTASH_REDIS_REST_TOKEN` ,
`NEXT_PUBLIC_SITE_URL` . Template: `.env.example` . Engine scripts load via `lib/engine/load-env.ts` .

Appendix C — Commands & Scripts

```
npm run dev            # Next dev server
npm run build          # production build
npm run typecheck      # tsc --noEmit
npm run lint           # next lint
# Engine (service-role; need .env.local):
npm run engine:discover # find new stores from broad ad search
npm run engine:inventory # scrape stock + snapshot + niche-tag + purge junk
npm run engine:classify # drain the untagged-niche queue (Groq)
npm run engine:ads      # scrape active Meta ads
npm run engine:winners  # recompute velocity + is_winner + lead_score
npm run stats           # DB health snapshot
npm run seed:list       # seed verified DZ store list
npm run seed:winner     # seed an engineered demo winner store
# Migrations: npx supabase db push --db-url <session-pooler-url>
# This doc:   node scripts/gen-docs-pdf.mjs    (md → pdf via Playwright)
```